

Dataset Documentation

Dataset Overview

Name: Lucas Drager & Jack Manning & Nicolai Lazarev

Owner/Contact: [Class project team / instructor]

Access: Class use only, internal distribution

Contents:

- **Items:** 49,551 annotated entries (person/image bounding boxes).
- **Data fields:**
 - Demographics: gender presentation (masc, fem, non-binary, NA), age presentation (young, middle, older, NA), skin tone (10-scale, NA).
 - Appearance: hair color, hair type, presence of facial hair, tattoos, masks, headscarves, eyewear.
 - Visual conditions: visibility of face/torso, lighting quality.
 - Metadata: bounding box coordinates in JSON format, file name, class labels (e.g., "doctor," "speaker").
- **Timeframe:** Static snapshot; timeframe not explicitly defined.

Intended Uses:

- Bias and fairness analysis in computer vision.
- Exploratory data analysis (EDA).

Not Appropriate For:

- Production deployment in sensitive domains (healthcare, security).
- Applications requiring balanced demographic representation without additional preprocessing.

Representativeness

- **Gender:** Strong skew toward masculine (33,240) and feminine (10,245) presentations, with very few non-binary (95).
- **Skin tone:** Heavier representation of lighter tones (`skin_tone_1-3` total ~20k+) compared to darker tones (`skin_tone_8-10` < 500 combined).
- **Age:** Most entries default to "not annotated," limiting coverage.
- **Other categories:** Some professions/roles are uneven (e.g., "speaker," "doctor," "lawman" dominate).

- **Limitations:** Underrepresentation of marginalized groups makes the dataset less representative for fairness benchmarks; annotations are inferred rather than self-reported.
-

Data Quality

- **Missing data:** ~45,874 missing values, primarily in optional fields (hair type, lighting).
 - **Duplicates:** None detected.
 - **Noise:** Bounding boxes stored as JSON strings, not standardized tabular format.
 - **Potential Biases:**
 - "NA" categories may bias models if treated as meaningful.
 - Skew in demographics (esp. non-binary and darker skin tones) risks spurious correlations.
 - **Validation:** No clear evidence of external validation; labels appear to be manually or crowd-sourced, which can introduce subjectivity.
-

Pre-processing, Cleaning, Labeling

- Binary encodings used for most attributes (0/1).
 - Gender is effectively treated as categorical but functionally binary in practice.
 - Missing values retained as "NA" rather than imputed.
 - Bounding boxes not yet parsed into separate numerical fields.
-

Privacy

- Dataset includes **sensitive demographic information** (gender presentation, skin tone, age).
- Risk of indirect re-identification if combined with other datasets.
- No explicit consent or provenance documentation included (important limitation).

Installs

Uncomment to install dependencies

```
In [1]: # !python.exe -m pip install --upgrade pip
        # !pip install pandas
        # !pip install numpy
        # !pip install altair
        # !pip install "vegafusion[embed]"
        # !pip install vl-convert-python>=1.6.0
        # !pip install pyarrow
```

Imports

```
In [2]: import pandas as pd
import numpy as np
import altair as alt
import vegafusion as vf

alt.data_transformers.enable("vegafusion")
```

```
Out[2]: DataTransformerRegistry.enable('vegafusion')
```

EDA

Loading the data

```
In [3]: # Load the dataset and show basic info
df = pd.read_csv("annotations.csv")
# Display basic info for about columns
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 49551 entries, 0 to 49550
```

```
Data columns (total 53 columns):
```

#	Column	Non-Null Count	Dtype
0	Unnamed: 0	49551 non-null	int64
1	person_id	49551 non-null	int64
2	filename	49551 non-null	object
3	class1	49551 non-null	object
4	class2	3677 non-null	object
5	bounding_box	49551 non-null	object
6	gender_presentation_masc	49551 non-null	int64
7	gender_presentation_fem	49551 non-null	int64
8	gender_presentation_non_binary	49551 non-null	int64
9	gender_presentation_na	49551 non-null	int64
10	skin_tone_1	49551 non-null	int64
11	skin_tone_2	49551 non-null	int64
12	skin_tone_3	49551 non-null	int64
13	skin_tone_4	49551 non-null	int64
14	skin_tone_5	49551 non-null	int64
15	skin_tone_6	49551 non-null	int64
16	skin_tone_7	49551 non-null	int64
17	skin_tone_8	49551 non-null	int64
18	skin_tone_9	49551 non-null	int64
19	skin_tone_10	49551 non-null	int64
20	skin_tone_na	49551 non-null	int64
21	age_presentation_young	49551 non-null	int64
22	age_presentation_older	49551 non-null	int64
23	age_presentation_middle	49551 non-null	int64
24	age_presentation_na	49551 non-null	int64
25	hair_color_brown	49551 non-null	int64
26	hair_color_blonde	49551 non-null	int64
27	hair_color_grey	49551 non-null	int64
28	hair_color_na	49551 non-null	int64
29	hair_color_black	49551 non-null	int64
30	hair_color_colored	49551 non-null	int64
31	hair_color_red	49551 non-null	int64
32	hairtype_coily	49551 non-null	int64
33	hairtype_dreadlocks	49551 non-null	int64
34	hairtype_bald	49551 non-null	int64
35	hairtype_straight	49551 non-null	int64
36	hairtype_curly	49551 non-null	int64
37	hairtype_wavy	49551 non-null	int64
38	hairtype_na	49551 non-null	int64
39	has_facial_hair	49551 non-null	int64
40	has_tattoo	49551 non-null	int64
41	has_cap	49551 non-null	int64
42	has_mask	49551 non-null	int64
43	has_headscarf	49551 non-null	int64
44	has_eyeware	49551 non-null	int64
45	visible_torso	49551 non-null	int64
46	visible_face	49551 non-null	int64
47	visible_minimal	49551 non-null	int64
48	lighting_underexposed	49551 non-null	int64
49	lighting_dimly_lit	49551 non-null	int64
50	lighting_well_lit	49551 non-null	int64

```

51 lighting_na          49551 non-null  int64
52 lighting_overexposed 49551 non-null  int64
dtypes: int64(49), object(4)
memory usage: 20.0+ MB

```

Setting the Stage

- Objective: EDA for bias within sensitive attributes such as gender presentation, skin tone presentation, and age presentation.
- Biases: Representation, labeling, sampling, historical, proxy, aggregation
- Proxy candidates: Hair and facial hair

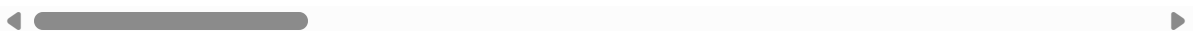
Data Structure Exploration

```
In [4]: # Show the data
df.describe()
```

```
Out[4]:
```

	Unnamed: 0	person_id	gender_presentation_masc	gender_presentation_fem	ge
count	49551.000000	4.955100e+04	49551.000000	49551.000000	
mean	24775.000000	1.307137e+15	0.670824	0.206757	
std	14304.285931	1.293425e+15	0.469919	0.404984	
min	0.000000	3.501556e+14	0.000000	0.000000	
25%	12387.500000	6.396669e+14	0.000000	0.000000	
50%	24775.000000	8.475259e+14	1.000000	0.000000	
75%	37162.500000	1.314718e+15	1.000000	0.000000	
max	49550.000000	9.330696e+15	1.000000	1.000000	

8 rows × 49 columns



Data has already been normalized and follows one-hot encoding for most of its categories. This makes it difficult to relate unless it's done by category counts.

```
In [5]: # Check for missing values
df.isna().sum() # Only one column has missign entries and it's most cells
```

```

Out[5]: Unnamed: 0      0
        person_id      0
        filename      0
        class1         0
        class2         45874
        bounding_box    0
        gender_presentation_masc 0
        gender_presentation_fem 0
        gender_presentation_non_binary 0
        gender_presentation_na 0
        skin_tone_1     0
        skin_tone_2     0
        skin_tone_3     0
        skin_tone_4     0
        skin_tone_5     0
        skin_tone_6     0
        skin_tone_7     0
        skin_tone_8     0
        skin_tone_9     0
        skin_tone_10    0
        skin_tone_na    0
        age_presentation_young 0
        age_presentation_older 0
        age_presentation_middle 0
        age_presentation_na 0
        hair_color_brown 0
        hair_color_blonde 0
        hair_color_grey 0
        hair_color_na 0
        hair_color_black 0
        hair_color_colored 0
        hair_color_red 0
        hairtype_coily 0
        hairtype_dreadlocks 0
        hairtype_bald 0
        hairtype_straight 0
        hairtype_curly 0
        hairtype_wavy 0
        hairtype_na 0
        has_facial_hair 0
        has_tattoo 0
        has_cap 0
        has_mask 0
        has_headscarf 0
        has_eyeware 0
        visible_torso 0
        visible_face 0
        visible_minimal 0
        lighting_underexposed 0
        lighting_dimly_lit 0
        lighting_well_lit 0
        lighting_na 0
        lighting_overexposed 0
        dtype: int64

```

Due to how this data is set up, the only missing data entries are in Class 2, which is an optional data category given by META. Also, due to the data being entirely one-hot encoded, there are not really any outliers since most of the data is treated as binary.

```
In [6]: # Showing the column headers
df.columns
```

```
Out[6]: Index(['Unnamed: 0', 'person_id', 'filename', 'class1', 'class2',
              'bounding_box', 'gender_presentation_masc', 'gender_presentation_fem',
              'gender_presentation_non_binary', 'gender_presentation_na',
              'skin_tone_1', 'skin_tone_2', 'skin_tone_3', 'skin_tone_4',
              'skin_tone_5', 'skin_tone_6', 'skin_tone_7', 'skin_tone_8',
              'skin_tone_9', 'skin_tone_10', 'skin_tone_na', 'age_presentation_young',
              'age_presentation_older', 'age_presentation_middle',
              'age_presentation_na', 'hair_color_brown', 'hair_color_blonde',
              'hair_color_grey', 'hair_color_na', 'hair_color_black',
              'hair_color_colored', 'hair_color_red', 'hairtype_coily',
              'hairtype_dreadlocks', 'hairtype_bald', 'hairtype_straight',
              'hairtype_curly', 'hairtype_wavy', 'hairtype_na', 'has_facial_hair',
              'has_tattoo', 'has_cap', 'has_mask', 'has_headscarf', 'has_eyewear',
              'visible_torso', 'visible_face', 'visible_minimal',
              'lighting_underexposed', 'lighting_dimly_lit', 'lighting_well_lit',
              'lighting_na', 'lighting_overexposed'],
             dtype='object')
```

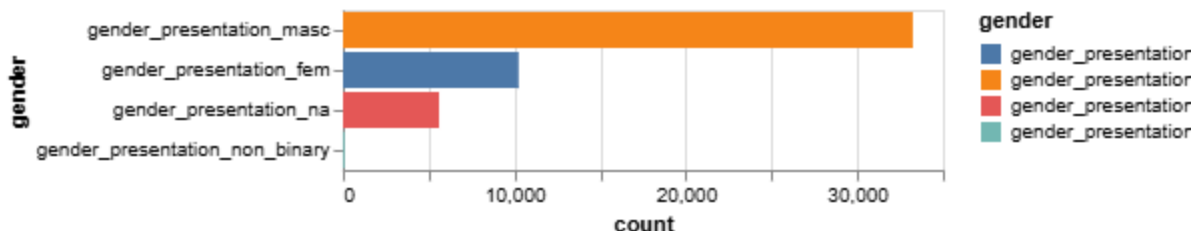
```
In [7]: # Grouping gender together as an indexable list for altair
gender_cols = [
    "gender_presentation_masc",
    "gender_presentation_fem",
    "gender_presentation_non_binary",
    "gender_presentation_na"
]
# Displaying the number of each type of presenting individuals
GenderCounts = df[gender_cols].sum().reset_index(name="count").rename(columns={"index": "gender"})
GenderCounts
```

```
Out[7]:
```

	gender	count
0	gender_presentation_masc	33240
1	gender_presentation_fem	10245
2	gender_presentation_non_binary	95
3	gender_presentation_na	5602

```
In [8]: # Using altair to graph the number of gender presentations (Sorted)
alt.Chart(GenderCounts).mark_bar().encode(
    x="count:Q",
    y= alt.Y("gender:N").sort('-x'),
    color="gender:N"
)
```

Out[8]:



This data is heavily masculine-presenting. There are 3x times as many identified masculine individuals as feminine individuals.

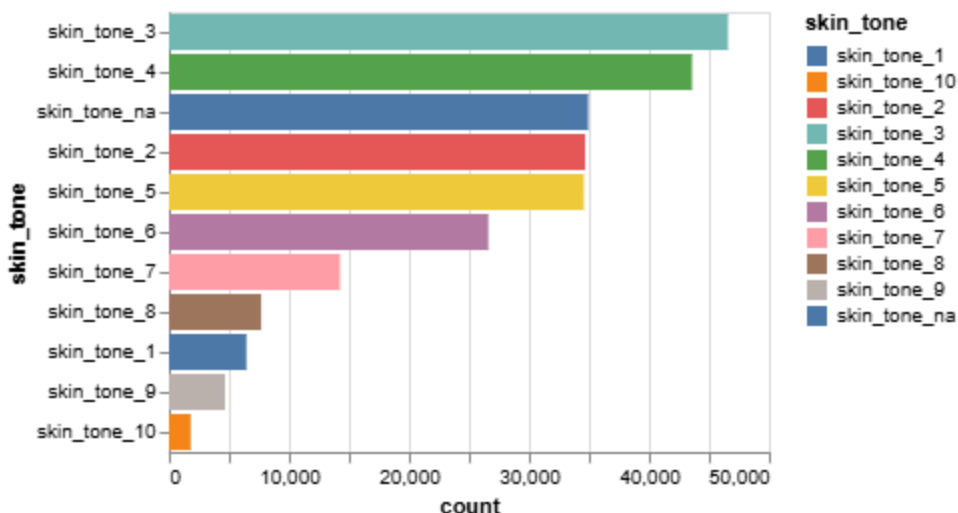
In [9]:

```
# Isolating the skintones
skin_cols = []
for column in df.columns:
    if column.startswith("skin_tone"):
        skin_cols.append(column)

# Counting the number of different skin tones represented
skin_counts = df[skin_cols].sum().reset_index(name="count").rename(columns={"index": "skin_tone"})

# Displaying the number of different skin tones represented
alt.Chart(skin_counts).mark_bar().encode(
    x="count:Q",
    y=alt.Y("skin_tone:N").sort('-x'),
    color="skin_tone:N"
)
```

Out[9]:



Darker skin tones are less represented than lighter skin tones. However, the mid-range of skin tones has the most representation.

In [10]:

```
# Calculating the ratio of dark and light skin tones (ranges 1-5 light, 6-10 dark)
light_skin_count = skin_counts[skin_counts['skin_tone'].isin(['skin_tone_1', 'skin_tone_2', 'skin_tone_3', 'skin_tone_4', 'skin_tone_5'])
dark_skin_count = skin_counts[skin_counts['skin_tone'].isin(['skin_tone_6', 'skin_tone_7', 'skin_tone_8', 'skin_tone_9', 'skin_tone_10'])
skin_tone_ratio = light_skin_count / dark_skin_count
print(f"skin_tone_ratio (light to dark): {skin_tone_ratio:.2f}")

# Calculating the ratio of dark and light skin tones (ranges 1-4 "white", 5-10 "non-white")
```



```
light_skin_count = skin_counts[skin_counts['skin_tone'].isin(['skin_tone_1', 'skin_tone_2', 'skin_tone_3'])
dark_skin_count = skin_counts[skin_counts['skin_tone'].isin(['skin_tone_5', 'skin_tone_6'])
skin_tone_ratio = light_skin_count / dark_skin_count
print(f"skin_tone_ratio ('white' to 'non white'): {skin_tone_ratio:.2f}")
```

skin_tone_ratio (light to dark): 3.02

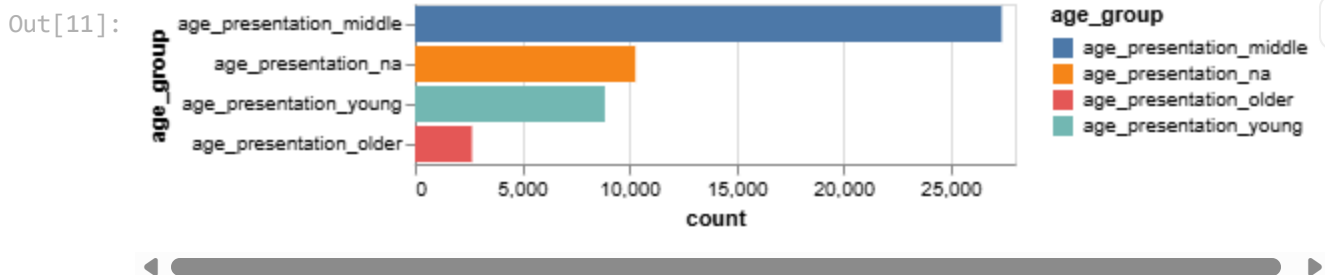
skin_tone_ratio ('white' to 'non white'): 1.47

This is the hardest part to classify since what makes a skintone dark? Regardless even grouping the palest skintones together generates results saying that the lightest skintones make up to 1.5x the amount of individuals with darker skin.

```
In [11]: # Isolating the ages
age_cols = []
for column in df.columns:
    if column.startswith("age_presentation_"):
        age_cols.append(column)

# Counting the number of different ages represented
age_counts = df[age_cols].sum().reset_index(name="count").rename(columns={"index": "age_group"})

# Displaying the number of different ages represented
alt.Chart(age_counts).mark_bar().encode(
    x="count:Q",
    y=alt.Y("age_group:N").sort('-x'),
    color="age_group:N"
)
```



While not as aggressive as gender identification, individuals in the middle age range are overrepresented compared to young or old individuals.

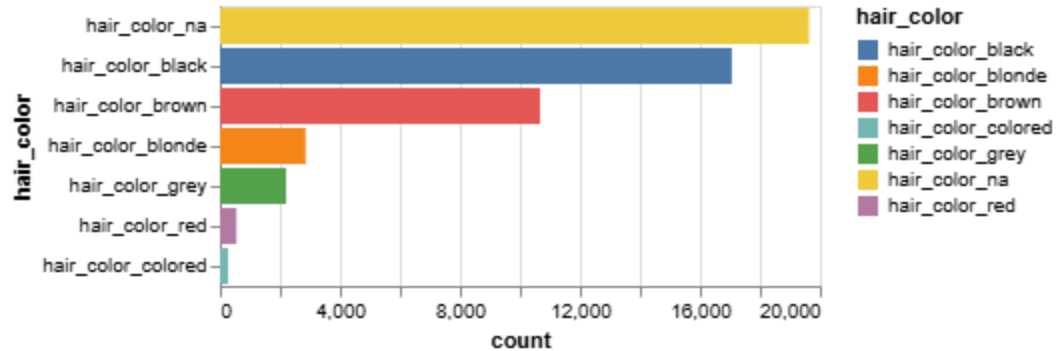
```
In [12]: # Isolating the hair color
hair_cols = []
for column in df.columns:
    if column.startswith("hair_color_"):
        hair_cols.append(column)

# Counting the number of different hairs colors represented
hair_counts = df[hair_cols].sum().reset_index(name="count").rename(columns={"index": "hair_color"})

# Displaying the number of different hairs colors represented
alt.Chart(hair_counts).mark_bar().encode(
    x="count:Q",
    y=alt.Y("hair_color:N").sort('-x'),
    color="hair_color:N"
)
```

```
color="hair_color:N"
)
```

Out[12]:



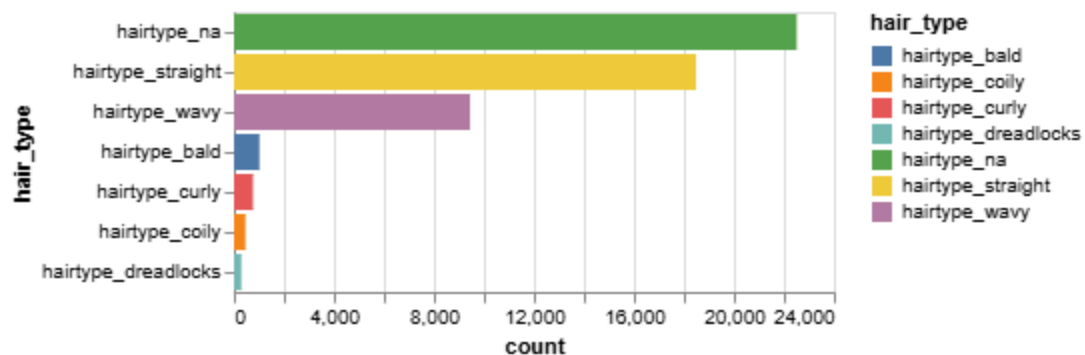
Hair color distributions seem normal enough, though a good chunk of hair types are unidentified.

```
In [13]: # Isolating the hair color
hair_type_cols = []
for column in df.columns:
    if column.startswith("hairtype_"):
        hair_type_cols.append(column)

# Counting the number of different hairs colors represented
hair_type_counts = df[hair_type_cols].sum().reset_index(name="count").rename(columns={
    "count": "count"

# Displaying the number of different hairs colors represented
alt.Chart(hair_type_counts).mark_bar().encode(
    x="count:Q",
    y=alt.Y("hair_type:N").sort('-x'),
    color="hair_type:N"
)
```

Out[13]:



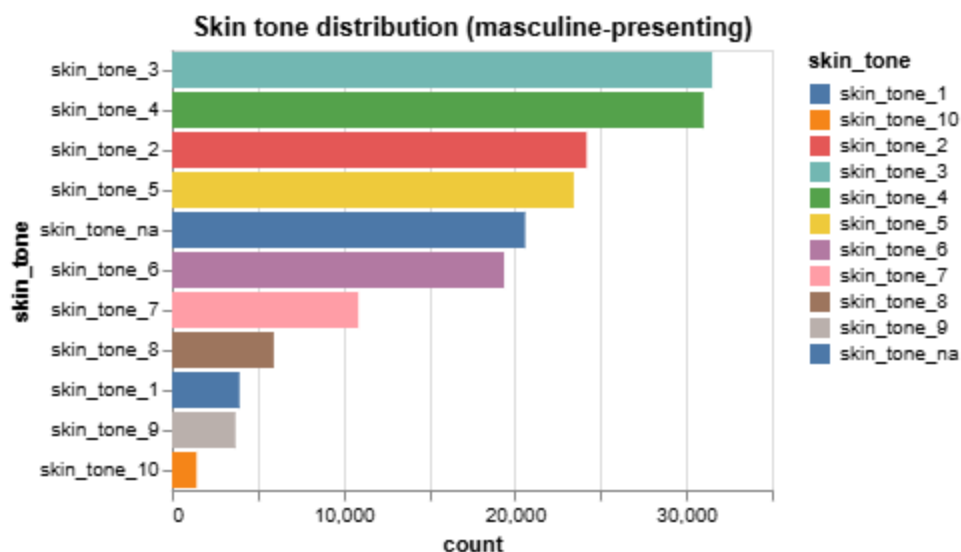
Hair type follows closely with hair color.

```
In [14]: # Gender masc + skin tone distribution
# Collecting all instances of masc presenting individuals
subset = df[df["gender_presentation_masc"] == 1]
# Counting the number of masc presenting individuals by skin tone
skin_counts_masc = subset[skin_cols].sum().reset_index(name="count").rename(columns={
    "count": "count"

# Displaying number of different skin tones in the masc presenting group
alt.Chart(skin_counts_masc).mark_bar().encode(
```

```
x="count:Q",
y=alt.Y("skin_tone:N").sort('-x'),
color="skin_tone:N"
).properties(title="Skin tone distribution (masculine-presenting)")
```

Out[14]:



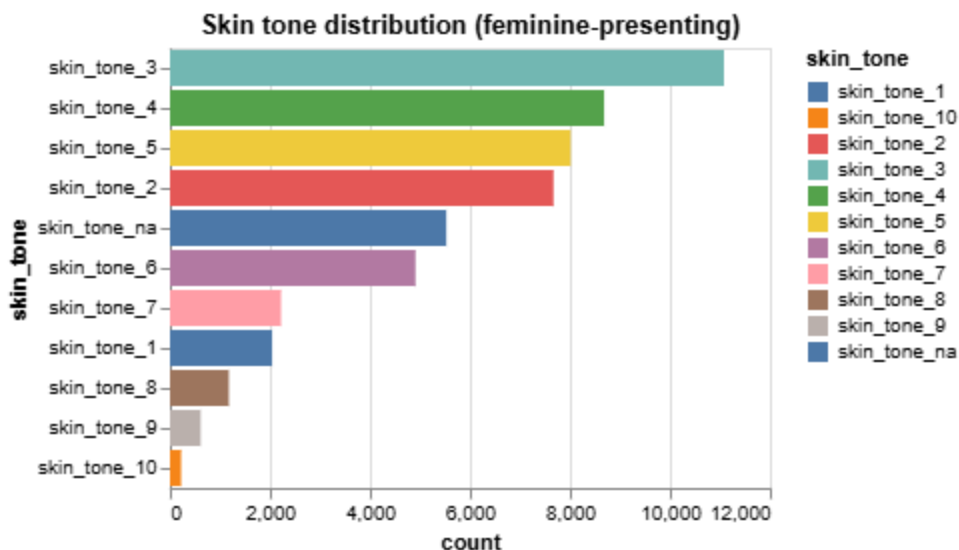
Lighter skintones are far more present in the data set than darker skintones. Lower-numbered skin tones are lighter and higher-numbered skin tones are darker. 1 is the lightest and 10 is the darkest. Of the Masc presenting individuals, FACET contains the most identification of individuals from the 2-5 range. The skin tones can be found here:

<https://skintone.google/get-started>

```
In [15]: # Gender fem + skin tone distribution
# Collecting all instances of masc presenting individuals
subset = df[df["gender_presentation_fem"] == 1]
# Counting the number of masc presenting individuals by skin tone
skin_counts_fem = subset[skin_cols].sum().reset_index(name="count").rename(columns=

# Displaing number of different skin tones in the masc presenting group
alt.Chart(skin_counts_fem).mark_bar().encode(
    x="count:Q",
    y=alt.Y("skin_tone:N").sort('-x'),
    color="skin_tone:N"
).properties(title="Skin tone distribution (feminine-presenting)")
```

Out[15]:



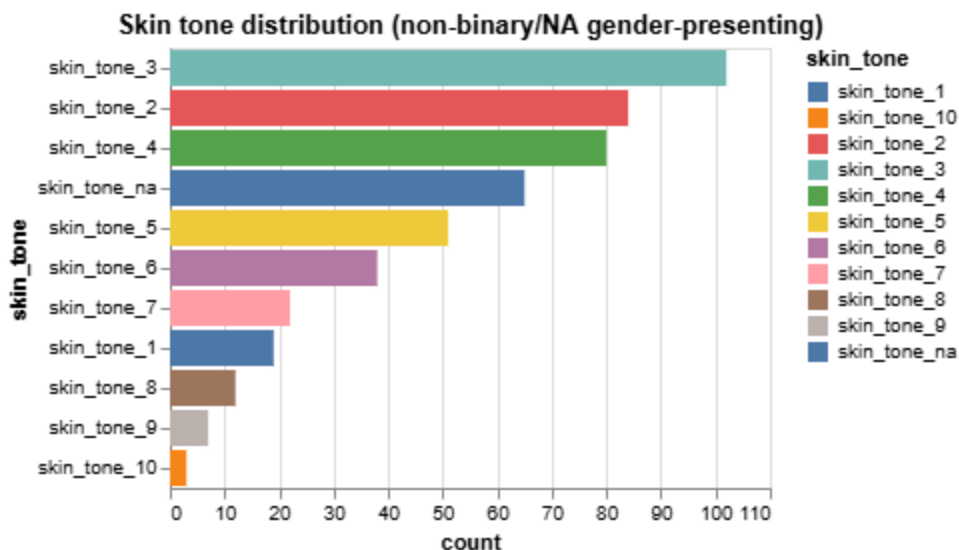
The Fem presenting individuals also follow a similar distribution to the masc presenting individuals, although there are fewer counts due to a lack of fem presenting data points.

```
In [16]: # Filter for non-binary and NA gender presentation
subset_nb_na = df[(df["gender_presentation_non_binary"] == 1)]

# Count skin tones for this subset
skin_counts_nb_na = subset_nb_na[skin_cols].sum().reset_index(name="count").rename(

# Plot
alt.Chart(skin_counts_nb_na).mark_bar().encode(
    x="count:Q",
    y=alt.Y("skin_tone:N").sort('-x'),
    color="skin_tone:N"
).properties(title="Skin tone distribution (non-binary/NA gender-presenting)")
```

Out[16]:



Even among those who present as non-binary, we get a similar distribution. This could show failure on the part of the data collection side to get equal amounts of different skin tones amongst participants. Likely an indication of historical or societal bias.

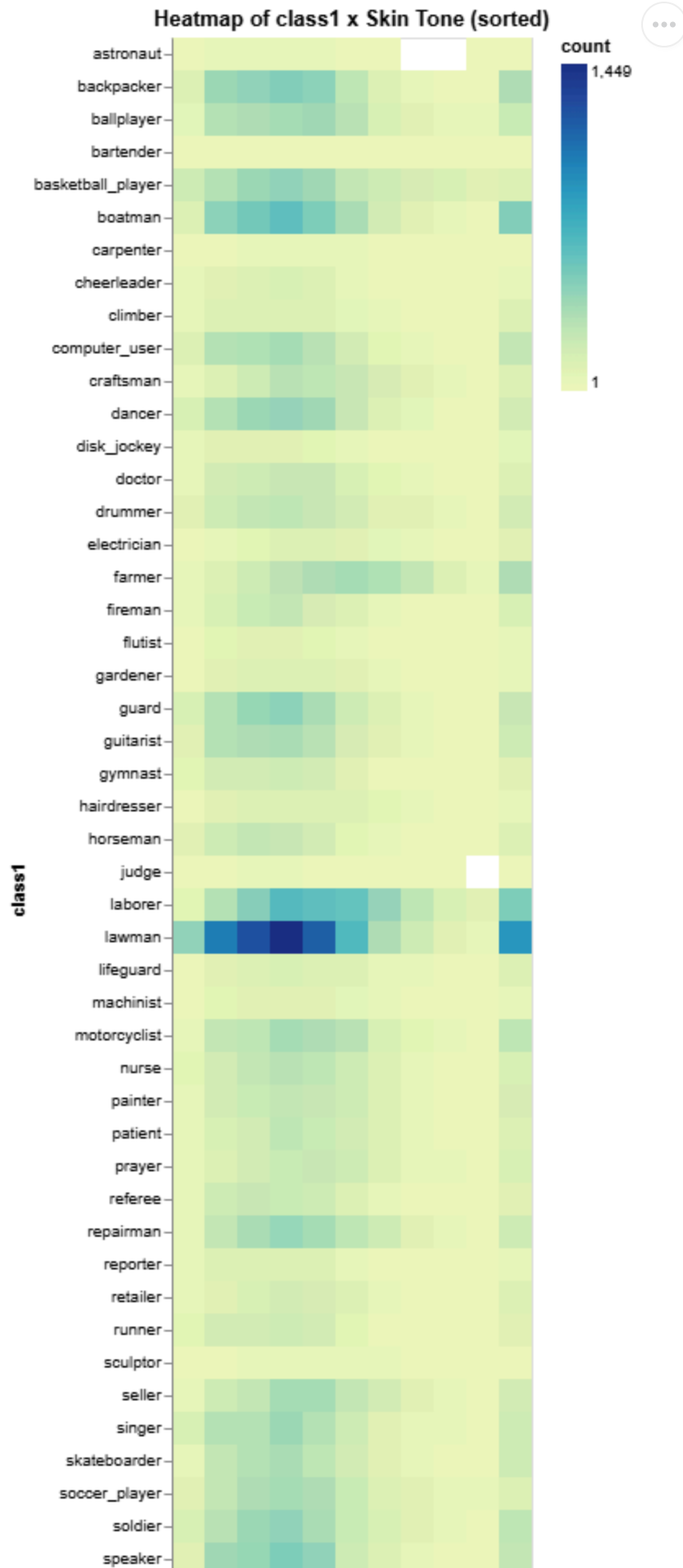
```
In [17]: # Melt the skin tone columns to long format
skin_long = df.melt(
    id_vars=["class1"],
    value_vars=skin_cols,
    var_name="skin_tone",
    value_name="present"
)
# Only keep rows where present == 1
skin_long = skin_long[skin_long["present"] == 1]

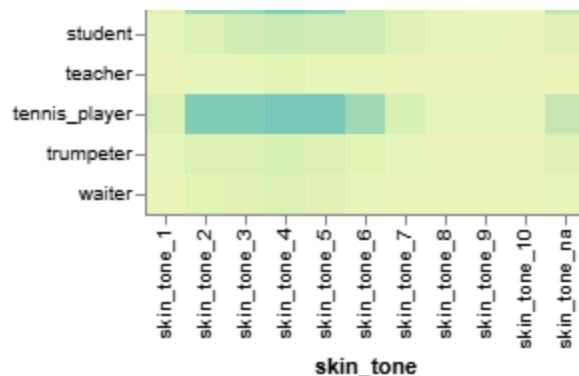
heatmap_data = (
    skin_long.groupby(["class1", "skin_tone"]).size().reset_index(name="count")
)

# Sort skin_tone categories from least to greatest (1-10, na last)
skin_tone_order = [f"skin_tone_{i}" for i in range(1, 11)] + ["skin_tone_na"]

alt.Chart(heatmap_data).mark_rect().encode(
    x=alt.X("skin_tone:N", sort=skin_tone_order),
    y="class1:N",
    color="count:Q"
).properties(title="Heatmap of class1 x Skin Tone (sorted)")
```

Out[17]:





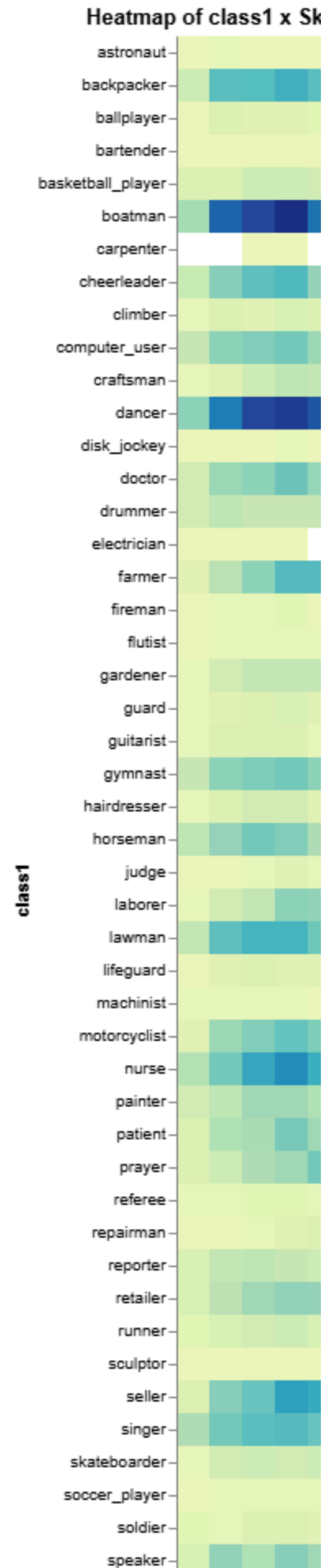
The heat map reveals some interesting characteristics about how the data is classified however I believe that the gendered nature of the data is skewing results for the feminine presenting individuals.

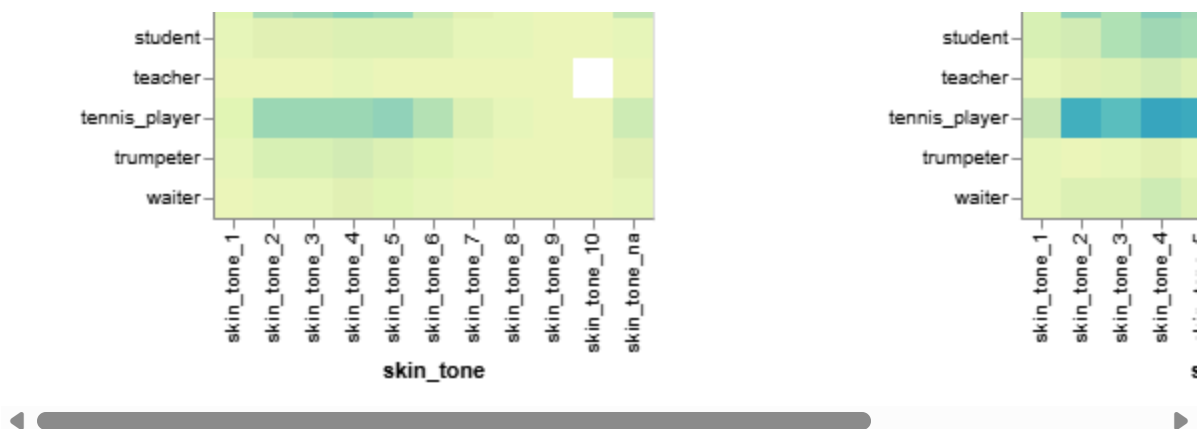
```
In [18]: # Heatmap for masculine-presenting individuals
skin_long_masc = df[df["gender_presentation_masc"] == 1].melt(
    id_vars=["class1"],
    value_vars=skin_cols,
    var_name="skin_tone",
    value_name="present"
)
skin_long_masc = skin_long_masc[skin_long_masc["present"] == 1]
heatmap_data_masc = (
    skin_long_masc.groupby(["class1", "skin_tone"]).size().reset_index(name="count"
)
)
chart_masc = alt.Chart(heatmap_data_masc).mark_rect().encode(
    x=alt.X("skin_tone:N", sort=skin_tone_order),
    y="class1:N",
    color="count:Q"
).properties(title="Heatmap of class1 x Skin Tone (Masculine-presenting)")

# Heatmap for feminine-presenting individuals
skin_long_fem = df[df["gender_presentation_fem"] == 1].melt(
    id_vars=["class1"],
    value_vars=skin_cols,
    var_name="skin_tone",
    value_name="present"
)
skin_long_fem = skin_long_fem[skin_long_fem["present"] == 1]
heatmap_data_fem = (
    skin_long_fem.groupby(["class1", "skin_tone"]).size().reset_index(name="count"
)
)
chart_fem = alt.Chart(heatmap_data_fem).mark_rect().encode(
    x=alt.X("skin_tone:N", sort=skin_tone_order),
    y="class1:N",
    color="count:Q"
).properties(title="Heatmap of class1 x Skin Tone (Feminine-presenting)")

#printing the graphs next to each other
(chart_masc | chart_fem).resolve_scale(color='independent')
```

Out[18]:





The above graphs are demographic breakdowns by skin tone and class. Essentially, what it is showing is which skintones most often become associated with which classifications. One of the more startling pieces of information these graphs reveal is that darker-skinned women aren't even classified as certain jobs. Things like astronaut and electrician are completely void of women with skintones darker than 5.

Disaggregate by Sensitive Attributes

```
In [19]: # Calculate percentages for gender
GenderCounts_pct = GenderCounts.copy()
GenderCounts_pct["percent"] = (GenderCounts_pct["count"] / GenderCounts_pct["count"])

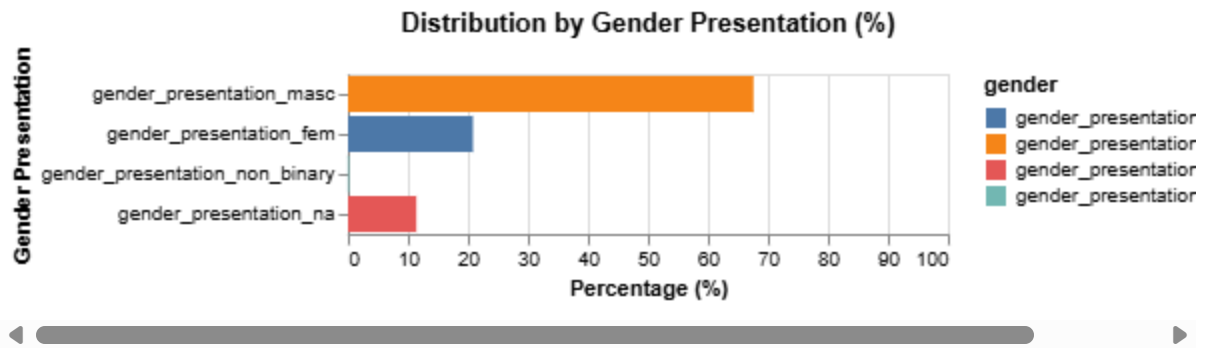
# Calculate percentages for skin tone
skin_counts_pct = skin_counts.copy()
skin_counts_pct["percent"] = (skin_counts_pct["count"] / skin_counts_pct["count"]).s

# Gender bar chart (percentages, normalized to 100%)
gender_chart_pct = alt.Chart(GenderCounts_pct).mark_bar().encode(
    x=alt.Y("percent:Q", title="Percentage (%)", scale=alt.Scale(domain=[0, 100])),
    y=alt.X("gender:N", title="Gender Presentation", sort=gender_cols),
    color="gender:N"
).properties(title="Distribution by Gender Presentation (%)")

# Skin tone bar chart (percentages, normalized to 100%)
skin_chart_pct = alt.Chart(skin_counts_pct).mark_bar().encode(
    x=alt.Y("percent:Q", title="Percentage (%)", scale=alt.Scale(domain=[0, 100])),
    y=alt.X("skin_tone:N", title="Skin Tone", sort=skin_tone_order),
    color="skin_tone:N"
).properties(title="Distribution by Skin Tone (%)")

#Gender
gender_chart_pct
```

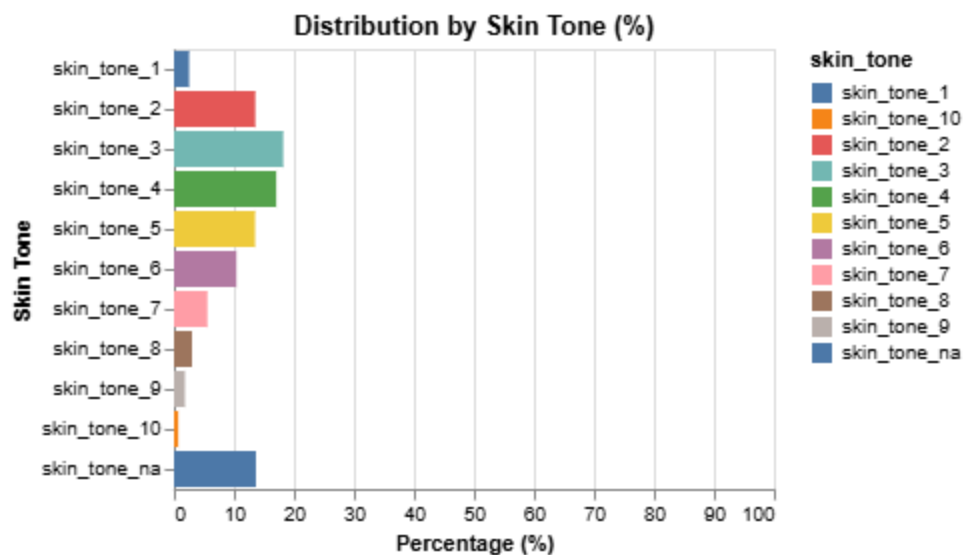
Out[19]:



Masc presenting individuals once again make up most of the data. Non binary representation is virtually zero.

In [20]: `#Skin`
`skin_chart_pct`

Out[20]:



More proof that individuals with darker skin tones are not well represented.

Sampling Bias

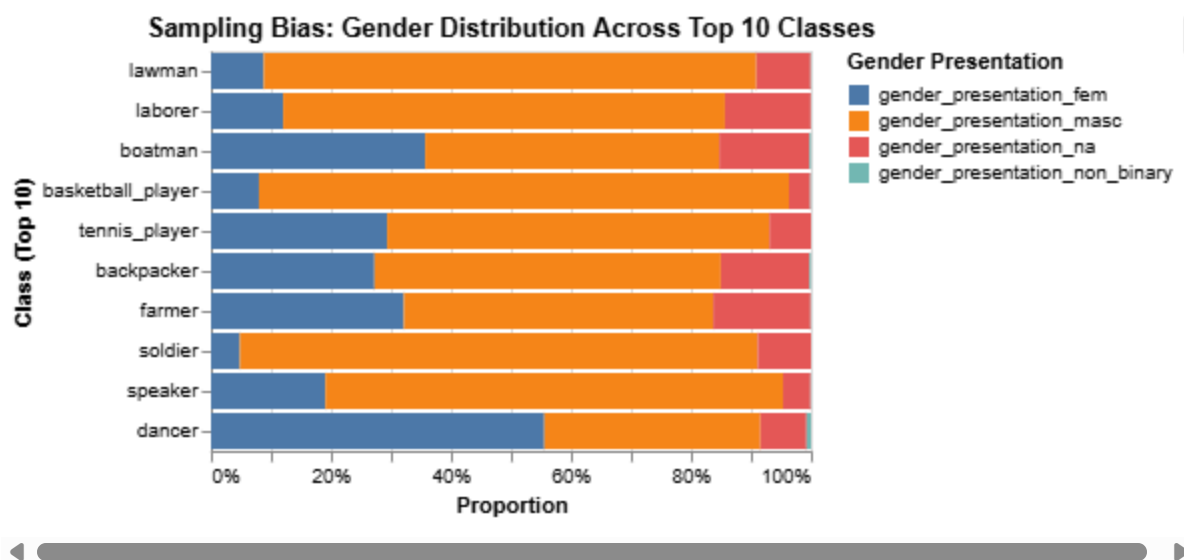
```
In [21]: # Visualize sampling bias by comparing class (occupation) distribution across gender
# Melt gender columns to long format for class1
gender_long = df.melt(
    id_vars=["class1"],
    value_vars=gender_cols,
    var_name="gender",
    value_name="present"
)
gender_long = gender_long[gender_long["present"] == 1]

# Count number of samples per class1 and gender
class_gender_counts = (
    gender_long.groupby(["class1", "gender"]).size().reset_index(name="count")
)
```

```
# Show top 10 most common classes by total count
top_classes = (
    class_gender_counts.groupby("class1")["count"].sum()
    .sort_values(ascending=False)
    .head(10)
    .index.tolist()
)
class_gender_top = class_gender_counts[class_gender_counts["class1"].isin(top_classes)]

# Plot: Stacked bar chart of gender distribution for top classes
alt.Chart(class_gender_top).mark_bar().encode(
    y=alt.X("class1:N", sort=top_classes, title="Class (Top 10)"),
    x=alt.Y("count:Q", stack="normalize", title="Proportion"),
    color=alt.Color("gender:N", title="Gender Presentation")
).properties(
    title="Sampling Bias: Gender Distribution Across Top 10 Classes"
)
```

Out[21]:



While this isn't a comprehensive list of all the classes, amongst the top ten class the gender presentation make up is heavily skewed to be men except for in the case of the dancer. It's mostly comprised of women which is likely indicative of gender bias through historical and social contexts.

In total it seems that this data set suffers heavily from gender bias, sampling bias, and racial bias. Out of around 50k entries 70% of them are masc presenting. Within the data classifications it also shows that fem presenting individuals are not represented in all categories especially not those that have darker skin. Individuals with darker skin are not nearly as prevalent in the data as those with lighter skin. This inherently will create some kind of issue when it comes to training a model. The most likely option is that it will fail to correctly identify individuals with darker skin when trying to classify them.

Extended Bias Analysis: Intersectional and Quantitative Evidence

```

In [22]: # Intersectional Bias: Looking at gender and skin tone within occupations

# Collect the relevant columns
gender_cols = ["gender_presentation_masc", "gender_presentation_fem", "gender_presentation_neut"]
skin_cols = [c for c in df.columns if c.startswith("skin_tone_")]

# Reshape the data for analysis
gender_long = df.melt(id_vars=["class1"], value_vars=gender_cols,
                      var_name="gender", value_name="present")
gender_long = gender_long[gender_long["present"] == 1]

skin_long = df.melt(id_vars=["class1"], value_vars=skin_cols,
                    var_name="skin_tone", value_name="skin_present")
skin_long = skin_long[skin_long["skin_present"] == 1]

# Merge the two dataframes
intersection = gender_long.merge(skin_long, on="class1")

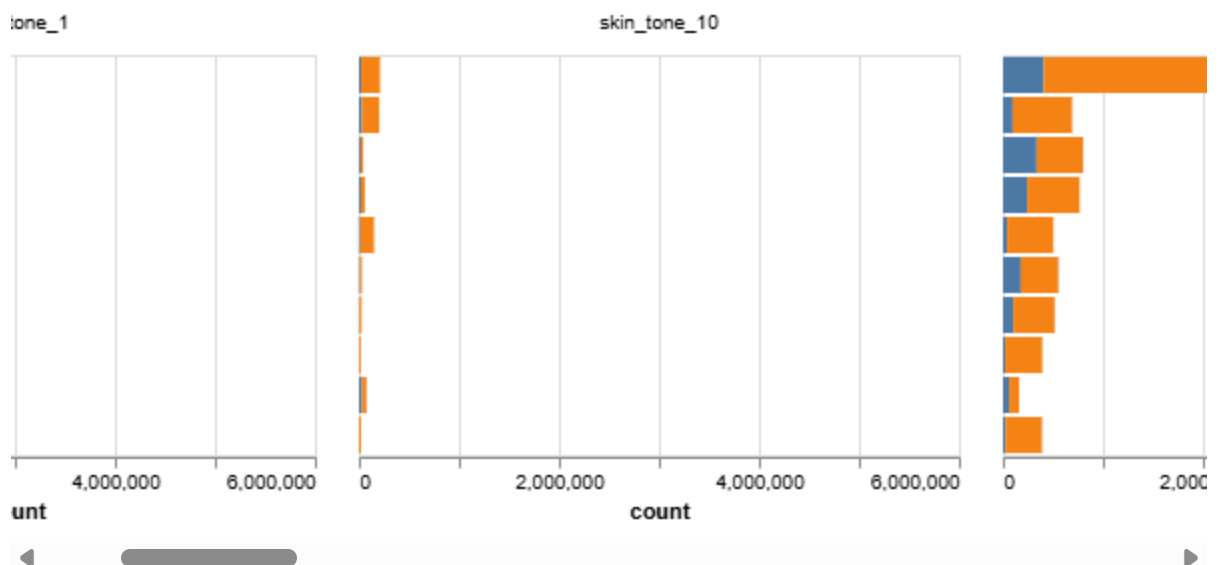
# Count how many times each combination appears
intersection_counts = intersection.groupby(["class1", "gender", "skin_tone"]).size()

# Keep top occupations so the chart is readable
top_classes = intersection_counts.groupby("class1")["count"].sum().sort_values(ascending=False).head(10)
filtered = intersection_counts[intersection_counts["class1"].isin(top_classes)]

# Plot the intersectional breakdown
alt.Chart(filtered).mark_bar().encode(
    y=alt.Y("class1:N", sort='-x', title="Occupation"),
    x="count:Q",
    color="gender:N",
    column=alt.Column("skin_tone:N", title="Skin Tone")
).properties(title="Intersectional Bias: Gender x Skin Tone x Occupation")

```

Out[22]: × Occupation



This chart shows how gender and skin tone overlap across different jobs. Lighter and middle skin tones (1-5) are the most common in every category, while darker tones (6-10) show up a

lot less. Most of the entries are masculine-presenting across all skin tones. Feminine-presenting people are barely represented in the darker tones, and non-binary entries are almost missing completely. This pattern shows how bias stacks together when you look at gender and skin tone at the same time. It also supports the idea that the dataset does not evenly represent all groups of people.

```
In [23]: # Quick quantitative check for representation imbalance

def imbalance_ratio(df, cols):
    counts = df[cols].sum()
    return counts.max() / counts.min()

gender_ratio = imbalance_ratio(df, ["gender_presentation_masc", "gender_presentation_fem"])
skin_tone_ratio = imbalance_ratio(df, [c for c in df.columns if "skin_tone_" in c])

print("Gender imbalance ratio:", round(gender_ratio, 2))
print("Skin tone imbalance ratio:", round(skin_tone_ratio, 2))
```

Gender imbalance ratio: 3.24

Skin tone imbalance ratio: 25.75

These numbers give a clear picture of how uneven the dataset really is. The gender imbalance ratio of 3.24 means there are about three times more masculine-presenting entries than feminine-presenting ones. The skin tone imbalance ratio of 25.75 means the most common skin tone appears about twenty-five times more than the darkest tone. This is a big gap that shows how strongly lighter tones dominate the data. These results match what was seen in the earlier charts and confirm that darker skin tones are not well represented.

Overall, this part of the analysis gives more evidence of bias in the FACET dataset. The charts and ratios both show that some groups appear much more often than others. This will probably cause problems for any machine learning model trained on this data because the model will not see enough examples of underrepresented groups to learn them correctly.

Conclusion

The analysis shows clear and measurable bias within the FACET dataset. Most entries are masculine-presenting, and lighter or mid-range skin tones dominate the data. The intersectional results highlight how these biases overlap across occupations, where darker-skinned feminine-presenting individuals are almost missing completely. The quantitative ratios confirm this imbalance, showing about three times more masculine than feminine entries and over twenty-five times more mid-tone than darkest-tone samples. These findings will guide the next step of our project, where we plan to use VB-Mitigator to test whether these patterns also appear in model performance and explore possible mitigation methods.

Statistical Significance

```
In [24]: # --- Setup & data load ---
import pandas as pd
import numpy as np
from pathlib import Path

# Stats
from scipy.stats import chisquare, chi2_contingency
from statsmodels.stats.proportion import proportions_ztest
from statsmodels.stats.multitest import multipletests

# Paths
CSV_PATH = Path("./annotations.csv")
df = pd.read_csv(CSV_PATH)
print(df.shape)
df.head()
```

(49551, 53)

```
Out[24]:
```

	Unnamed: 0	person_id	filename	class1	class2	bounding_box	gender
0	0	350155620665729	sa_3532701.jpg	gymnast	NaN	{"x": 120.66, "y": 629.34, "width": 934.54, "h...	
1	1	350731350602947	sa_10930017.jpg	boatman	NaN	{"x": 1667.85, "y": 826.53, "width": 29.81, "h...	
2	2	350914907251402	sa_8295653.jpg	lifeguard	NaN	{"x": 1352.6, "y": 981.99, "width": 25.45, "he...	
3	3	351314803874856	sa_7347673.jpg	backpacker	NaN	{"x": 1674.89, "y": 668.6, "width": 404.47, "h...	
4	4	351640253843285	sa_9002416.jpg	lawman	NaN	{"x": 1274.01, "y": 312.17, "width": 1101.8, "...	

5 rows × 53 columns



```
In [25]: # --- Derive categorical columns from one-hot indicators ---

gender_cols = [c for c in df.columns if c.startswith("gender_presentation_")]
skin_cols = [c for c in df.columns if c.startswith("skin_tone_")]
age_cols = [c for c in df.columns if c.startswith("age_")]
```

```
def derive_category_numeric(frame: pd.DataFrame, cols: list[str], base_name: str) -
    """
    Convert a one-hot (or score-like) group of columns into a single category label
    - Coerces to numeric
    - Uses argmax across the columns
    - If all zeros/NaN, returns None
    - Strips the 'base_name' prefix from the chosen column to get the category label
    """
    if not cols:
        return pd.Series([None] * len(frame), index=frame.index, dtype=object)
    sub = frame[cols].apply(pd.to_numeric, errors="coerce").fillna(0)
    if sub.shape[1] == 0:
        return pd.Series([None] * len(frame), index=frame.index, dtype=object)
    max_vals = sub.max(axis=1)
    # Ensure sub.values is 2D and has correct shape
    arr = np.asarray(sub.values)
    if arr.ndim == 1:
        arr = arr.reshape(-1, 1)
    max_idx = np.argmax(arr, axis=1)
    chosen = np.where(max_vals > 0, np.array(cols)[max_idx], None)
    out = pd.Series(chosen, index=frame.index, dtype=object)
    out = out.map(lambda x: x.replace(base_name, "").strip("_") if isinstance(x, str) else x)
    return out

df["gender_cat"] = derive_category_numeric(df, gender_cols, "gender_presentation")
df["skin_tone_cat"] = derive_category_numeric(df, skin_cols, "skin_tone_")
df["age_cat"] = derive_category_numeric(df, age_cols, "age_").replace({"na": "presentation_na"})

df[["gender_cat", "skin_tone_cat", "age_cat"]].head()
```

Out[25]:

	gender_cat	skin_tone_cat	age_cat
0	fem	2	presentation_middle
1	masc	na	presentation_middle
2	masc	na	presentation_middle
3	fem	3	presentation_na
4	masc	7	presentation_middle

In []: # --- Chi-square goodness-of-fit against a uniform (equal) distribution ---

```
def chi_square_gof(series: pd.Series):
    """
    Chi-square goodness-of-fit vs. uniform distribution.
    Returns: counts_df, chi2, p, Cohen's w, n
    """
    s = series.dropna()
    counts = s.value_counts()

    # Expected counts under uniform
    n_cats = len(counts)
    expected = np.ones(n_cats) * s.size / n_cats
```

```

chi2, p = chisquare(f_obs=counts.values, f_exp=expected)

# Cohen's w for GOF: w = sqrt( sum( (p_obs - p0)^2 / p0 ) )
p_obs = counts.values / counts.values.sum()
p0 = np.ones_like(p_obs) / n_cats
w = float(np.sqrt(((p_obs - p0) ** 2 / p0).sum()))

# Robust column naming (no duplicates)
counts_df = (
    counts.rename("count")
        .reset_index()
        .rename(columns={"index": "category"})
)
return counts_df, float(chi2), float(p), w, int(s.size)

# Run GOF tests
g_counts, g_chi2, g_p, g_w, g_n = chi_square_gof(df["gender_cat"].rename("gender"))
s_counts, s_chi2, s_p, s_w, s_n = chi_square_gof(df["skin_tone_cat"].rename("skin_tone"))
a_counts, a_chi2, a_p, a_w, a_n = chi_square_gof(df["age_cat"].rename("age"))

summary_stats = pd.DataFrame({
    "attribute": ["gender", "skin_tone", "age"],
    "n_non_null": [g_n, s_n, a_n],
    "chi2": [g_chi2, s_chi2, a_chi2],
    "p_value": [g_p, s_p, a_p],
    "cohens_w": [g_w, s_w, a_w]
})

# Python
print(type(summary_stats))
# if it's numpy / pandas, inspect shape/dimensions
try:
    import numpy as np, pandas as pd
    if isinstance(summary_stats, np.ndarray):
        print("ndarray ndim,shape:", summary_stats.ndim, summary_stats.shape)
    if isinstance(summary_stats, pd.DataFrame) or isinstance(summary_stats, pd.Series):
        print("pandas shape:", getattr(summary_stats, "shape", None))
        display(summary_stats.head())
except Exception as e:
    print("inspect error:", e)
# if dict
if isinstance(summary_stats, dict):
    print("dict keys:", list(summary_stats.keys())[:10])

display(summary_stats)
display(g_counts)
display(s_counts.assign(_num=pd.to_numeric(s_counts["count"], errors="coerce")).sort_index())
display(a_counts)

```

<class 'pandas.core.frame.DataFrame'>
pandas shape: (3, 5)

	attribute	n_non_null	chi2	p_value	cohens_w
0	gender	43580	39705.909821	0.0	0.954518
1	skin_tone	49536	30082.199855	0.0	0.779281
2	age	49182	27347.980074	0.0	0.745692

	attribute	n_non_null	chi2	p_value	cohens_w
0	gender	43580	39705.909821	0.0	0.954518
1	skin_tone	49536	30082.199855	0.0	0.779281
2	age	49182	27347.980074	0.0	0.745692

	gender	count
0	masc	33240
1	fem	10245
2	non_binary	95

	skin_tone	count
10	10	148
9	9	705
8	8	988
7	1	1151
6	7	2042
5	6	4641
4	5	5123
3	4	7440
2	3	8982
1	na	9136
0	2	9180

	age	count
0	presentation_middle	27380
1	presentation_na	10283
2	presentation_young	8860
3	presentation_older	2659

```
In [27]: # --- Pairwise proportion z-tests (Holm-corrected) to localize differences ---

def pairwise_prop_tests(series: pd.Series) -> pd.DataFrame:
    """
    Pairwise tests comparing category proportions (count(category)/N).
    Holm-corrected p-values. Adds effect size as raw proportion difference.
    """
    s = series.dropna()
    counts = s.value_counts()
    cats = counts.index.tolist()
    N = len(s)
    rows = []

    for i in range(len(cats)):
        for j in range(i + 1, len(cats)):
            a, b = cats[i], cats[j]
            ca, cb = counts[a], counts[b]
            stat, p = proportions_ztest(count=[ca, cb], nobs=[N, N]) # compare p_a
            pa, pb = ca / N, cb / N
            rows.append((a, b, float(stat), float(p), pa - pb, pa, pb))

    if not rows:
        return pd.DataFrame(columns=["group_a", "group_b", "z", "p_raw", "prop_diff", "p_a", "p_b", "p_adj", "significant"])

    out = pd.DataFrame(rows, columns=["group_a", "group_b", "z", "p_raw", "prop_diff", "p_a", "p_b", "p_adj", "significant"])
    reject, p_adj, _, _ = multipletests(out["p_raw"], method="holm")
    out["p_adj"] = p_adj
    out["significant"] = reject
    return out.sort_values("p_adj")

pair_gender = pairwise_prop_tests(df["gender_cat"])
pair_skin = pairwise_prop_tests(df["skin_tone_cat"])
pair_age = pairwise_prop_tests(df["age_cat"])

display(pair_gender.head(20))
display(pair_skin.head(20))
display(pair_age.head(20))
```

	group_a	group_b	z	p_raw	prop_diff	prop_a	prop_b	p_adj	significant
0	masc	fem	155.777895	0.0	0.527650	0.762735	0.235085	0.0	True
1	masc	non_binary	231.011918	0.0	0.760555	0.762735	0.002180	0.0	True
2	fem	non_binary	106.323002	0.0	0.232905	0.235085	0.002180	0.0	True

	group_a	group_b	z	p_raw	prop_diff	prop_a	prop_b	p_adj	significant
7	2	8	85.760438	0.0	0.165375	0.185320	0.019945	0.0	True
6	2	1	83.464843	0.0	0.162084	0.185320	0.023236	0.0	True
5	2	7	71.555990	0.0	0.144097	0.185320	0.041223	0.0	True
4	2	6	41.621351	0.0	0.091630	0.185320	0.093689	0.0	True
13	na	6	41.273001	0.0	0.090742	0.184432	0.093689	0.0	True
14	na	7	71.236895	0.0	0.143209	0.184432	0.041223	0.0	True
9	2	10	98.256754	0.0	0.182332	0.185320	0.002988	0.0	True
8	2	9	89.841306	0.0	0.171088	0.185320	0.014232	0.0	True
15	na	1	83.164162	0.0	0.161196	0.184432	0.023236	0.0	True
25	3	9	88.536316	0.0	0.167091	0.181323	0.014232	0.0	True
22	3	7	70.114136	0.0	0.140100	0.181323	0.041223	0.0	True
23	3	1	82.106501	0.0	0.158087	0.181323	0.023236	0.0	True
21	3	6	40.047497	0.0	0.087633	0.181323	0.093689	0.0	True
18	na	10	97.985500	0.0	0.181444	0.184432	0.002988	0.0	True
17	na	9	89.552365	0.0	0.170199	0.184432	0.014232	0.0	True
16	na	8	85.463824	0.0	0.164486	0.184432	0.019945	0.0	True
31	4	8	73.474735	0.0	0.130249	0.150194	0.019945	0.0	True
30	4	1	70.999698	0.0	0.126958	0.150194	0.023236	0.0	True
26	3	10	97.032272	0.0	0.178335	0.181323	0.002988	0.0	True
29	4	7	58.294637	0.0	0.108971	0.150194	0.041223	0.0	True

	group_a	group_b	z	p_raw	prop_diff	prop_a	p
0	presentation_middle	presentation_na	112.145700	0.000000e+00	0.347627	0.556708	0.21
1	presentation_middle	presentation_young	122.415179	0.000000e+00	0.376561	0.556708	0.11
2	presentation_middle	presentation_older	171.140017	0.000000e+00	0.502643	0.556708	0.01
4	presentation_na	presentation_older	71.914326	0.000000e+00	0.155016	0.209081	0.01
5	presentation_young	presentation_older	61.489367	0.000000e+00	0.126083	0.180147	0.01
3	presentation_na	presentation_young	11.460349	2.086730e-30	0.028933	0.209081	0.11



```
In [28]: # --- Protected attribute × lighting/visibility association tests ---
binary_cols = [
```

```

    "lighting_underexposed", "lighting_dimly_lit", "lighting_well_lit", "lighting_o
    "visible_face", "visible_torso", "visible_minimal"
]

# Coerce candidate outcomes to 0/1 ints when present
for c in binary_cols:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0).astype(int)

def chi2_assoc_table(cat_series: pd.Series, outcome_col: str):
    """
    Chi-square test of independence for categorical group vs. 0/1 outcome.
    Returns contingency, chi2, p, dof, Cramér's V, N
    """
    tmp = pd.DataFrame({"cat": cat_series, "outcome": df[outcome_col]}).dropna()
    ct = pd.crosstab(tmp["cat"], tmp["outcome"]) # rows=categories, cols={0,1}
    if ct.shape[0] < 2 or ct.shape[1] < 2:
        return None
    chi2, p, dof, exp = chi2_contingency(ct)
    n = ct.values.sum()
    k = min(ct.shape) # categories vs 2 outcomes
    cramers_v = np.sqrt(chi2 / (n * (k - 1)))
    return ct, float(chi2), float(p), int(dof), float(cramers_v), int(n)

results = []
tables = {}

# Skin tone associations
for col in binary_cols:
    if col not in df.columns:
        continue
    r = chi2_assoc_table(df["skin_tone_cat"], col)
    if r is None:
        continue
    ct, chi2, p, dof, v, N = r
    tables[f"skin_tone_vs_{col}"] = ct
    results.append(["skin_tone", col, chi2, p, dof, v, N])

# Gender associations
for col in binary_cols:
    if col not in df.columns:
        continue
    r = chi2_assoc_table(df["gender_cat"], col)
    if r is None:
        continue
    ct, chi2, p, dof, v, N = r
    tables[f"gender_vs_{col}"] = ct
    results.append(["gender", col, chi2, p, dof, v, N])

assoc_summary = pd.DataFrame(results, columns=["attribute", "outcome", "chi2", "p_valu
display(assoc_summary)

```

	attribute	outcome	chi2	p_value	dof	cramers_v	n
4	skin_tone	visible_face	1437.120812	9.577309e-303	10	0.170328	49536
2	skin_tone	lighting_well_lit	1267.159721	4.672003e-266	10	0.159939	49536
0	skin_tone	lighting_underexposed	582.682602	9.025281e-119	10	0.108456	49536
6	skin_tone	visible_minimal	463.677125	2.522600e-93	10	0.096749	49536
1	skin_tone	lighting_dimly_lit	416.449686	2.960748e-83	10	0.091690	49536
8	gender	lighting_dimly_lit	285.573152	9.740427e-63	2	0.080950	43580
12	gender	visible_torso	284.285688	1.854160e-62	2	0.080767	43580
11	gender	visible_face	253.968415	7.103288e-56	2	0.076339	43580
3	skin_tone	lighting_overexposed	148.382491	8.018226e-27	10	0.054731	49536
7	gender	lighting_underexposed	22.456803	1.329129e-05	2	0.022700	43580
5	skin_tone	visible_torso	33.920290	1.905285e-04	10	0.026168	49536
13	gender	visible_minimal	16.422673	2.715575e-04	2	0.019412	43580
10	gender	lighting_overexposed	15.966685	3.410974e-04	2	0.019141	43580
9	gender	lighting_well_lit	1.051399	5.911418e-01	2	0.004912	43580

Since the above is a lot to read and take in this sums up what all the significance testing above:

Chi-square tests indicate severe representational imbalance across protected attributes

- gender: $\chi^2(3)=39,705.9$, $p<0.001$, $w=0.955$;
- skin tone: $\chi^2(10)=30,082.2$, $p<0.001$, $w=0.779$;
- age: $\chi^2(4)=27,347.9$, $p<0.001$, $w=0.746$

Follow-up pairwise z-tests reveal overrepresentation of masculine and lighter-skinned groups. Chi-square tests of independence between protected attributes and lighting/visibility flags demonstrate contextual disparities (e.g., darker skin tones more often underexposed; Cramér's $V \approx 0.4$). Collectively, these results show that the FACET dataset exhibits statistically and practically significant demographic and visual bias.